Subscription Leak Finder

Architecture, Plan & Roadmap

July 2026

Subscription Leak Finder

A self-hosted, privacy-first tool that finds forgotten subscriptions draining your money — using a local LLM, zero paid AI APIs, and free-tier infrastructure.

---

1. Problem Statement

Most people are paying for subscriptions they forgot about, don't use anymore, or didn't realize would auto-renew. Subscription trackers exist, but they either:

- Require manually entering every subscription (tedious, gets abandoned), or
- Require connecting your bank account to a third-party SaaS (privacy risk, often paid)

Our approach: Forward subscription/receipt emails to a dedicated inbox. A local AI model reads each email and extracts the merchant, amount, and renewal date automatically. A dashboard shows total monthly burn and flags subscriptions you haven't touched in a while.

---

2. Core Principles

1. **$0 until deployment.** Every tool used is free or open-source. The only real cost is a small VPS (~$4–5/month), and only once we're ready to deploy — not during development.
2. **Local-first development.** Build and test the entire pipeline on your own machine before touching any cloud service. This keeps the feedback loop fast and avoids debugging deployment and logic at the same time.
3. **Privacy by design.** No email content or financial data ever touches a third-party AI API. Extraction happens locally via Ollama.
4. **Incremental delivery.** Each step in the roadmap is a working, testable slice of the product — not a big-bang build.

---

3. Tech Stack

Layer	Tool	Cost	Why
Frontend	Next.js	Free (Vercel free tier)	Modern React framework, great free hosting
Backend API	Fastify	Free (Railway free tier)	Lightweight, fast, strong TypeScript support

Database	Neon (Postgres)	Free tier	Serverless Postgres, generous free tier, same DB in dev and prod
ORM	Drizzle	Free	Lightweight, excellent TypeScript inference, plays well with Neon
AI Extraction	Ollama + qwen3:4b	Free (self-hosted)	Runs locally, no per-request cost, keeps data private
Automation	n8n	Free (self-hosted via Docker)	Handles email polling, scheduling, and notifications without custom glue code
Email intake	Gmail + App Password (IMAP)	Free	Simple, reliable way to receive forwarded receipts
Notifications	Telegram Bot	Free	Easiest free push-notification channel
Container runtime	Docker	Free	Runs Ollama + n8n consistently in dev and on the VPS
Hosting (final deploy only)	Hetzner VPS (CX22)	~$4–5/month	Only always-on cost; needed to keep Ollama + n8n running 24/7

---

4. Architecture Overview

4.1 Local Development (Phases 1–5)

4.2 Production (Phase 6 onward)

```
Next.js  → deployed to Vercel (free)
Fastify  → deployed to Railway (free tier)
Neon     → already cloud-hosted (no change)
```

```
Ollama   → moved to Hetzner VPS (Docker)
n8n      → moved to Hetzner VPS (Docker)
```

Only Ollama and n8n need an always-on machine, since they must run continuously in the background. Vercel and Railway remain free because Next.js and Fastify don't need to run locally 24/7 in the same way.

---

5. Data Flow (End-to-End)

1. User forwards a subscription/receipt email to a dedicated Gmail inbox.
2. n8n polls that inbox on a schedule (e.g., every 15 minutes) via IMAP.
3. n8n sends the raw email text to a Fastify endpoint: `POST /api/extract`.
4. Fastify calls Ollama (via the AI SDK + `ai-sdk-ollama` provider) with a structured prompt, asking the model to return JSON: `{ merchant, amount, currency, renewal_date, category }`.
5. Fastify validates and inserts the structured result into the `subscriptions` table in Neon Postgres.
6. The Next.js dashboard reads from Fastify's API and displays:
   - Total monthly burn
   - Upcoming renewals
   - Subscriptions unused/unmentioned for 60+ days
7. A separate n8n workflow runs weekly, queries the Fastify API for a summary, and sends it to the user via Telegram.

---

6. Roadmap (Step-by-Step, Commit-Based)

Each numbered item below is intended to be one commit and one working increment.

Phase 1 — Foundation

1. `chore: scaffold monorepo structure (frontend, backend)`
2. `feat: setup Neon Postgres and initial schema (subscriptions table)`
3. `feat: setup Fastify backend with health check endpoint`
4. `feat: connect Fastify to Neon via Drizzle ORM`

Phase 2 — Core CRUD (prove the pipeline manually before adding AI)

5. `feat: API endpoint to manually create a subscription entry`
6. `feat: API endpoint to list/filter subscriptions`
7. `feat: Next.js dashboard - basic table view of subscriptions`
8. `feat: Next.js dashboard - total monthly burn calculation + stats cards`

Phase 3 — AI Extraction (Ollama, fully local)

9. `chore: verify Ollama + qwen3:4b running locally`
10. `feat: Fastify endpoint that extracts structured subscription data from raw email text via Ollama`
11. `test: validate extraction accuracy with sample receipt emails`

Phase 4 — Automation (n8n, Docker, local)

12. `chore: install Docker and set up n8n locally`
13. `feat: n8n workflow - poll Gmail inbox for new emails`
14. `feat: n8n workflow - forward email content to Fastify extraction endpoint`
15. `feat: n8n workflow - insert extracted data into Postgres via Fastify`

Phase 5 — Intelligence Layer

16. `feat: flag subscriptions unused/unmentioned in 60+ days`
17. `feat: n8n workflow - weekly Telegram summary of burn + upcoming renewals`

Phase 6 — Production Hardening & Deployment (only phase with real cost)

18. `feat: add authentication (dashboard should not be public)`
19. `chore: environment variable management + secrets`
20. `chore: error handling + logging (Fastify + n8n)`
21. `chore: provision Hetzner VPS + deploy Ollama and n8n via Docker`
22. `chore: deploy Fastify to Railway, Next.js to Vercel`
23. `docs: README with architecture diagram + setup instructions`

---

7. Key Recommendations

- **Build CRUD before AI.** Get Next.js → Fastify → Neon working manually first. Adding AI extraction on top of a proven pipeline is much easier to debug than building both at once.
- **Use Drizzle, not Prisma**, for this project — lighter weight, strong TypeScript inference, and a clean fit with Neon's serverless driver.
- **Keep Neon in the cloud even during local development.** It's free and identical to production, so there's no separate "local Postgres" setup to maintain.
- **Start with a small model (qwen3:4b).** It's fast on modest hardware and reliable enough for structured JSON extraction tasks like this. Upgrade later only if extraction accuracy demands it.
- **Delay the VPS purchase.** Don't provision Hetzner until Phases 1–5 are working end-to-end locally. This avoids paying for infrastructure before you know it's needed.
- **Use npm workspaces, not Turborepo/Nx**, for the monorepo — this project doesn't need that level of tooling complexity.

---

8. Current Status

Item	Status
Ollama installed locally	✅ Done (qwen3:4b)
Node.js	✅ Installed
Git	✅ Installed
Docker	☐ Needed before Phase 4
Neon account	☐ Needed for Phase 1
Monorepo scaffold	☐ Next step

This document is a living plan — it will be updated as decisions are made and phases are completed.